

Стать IT-шником может каждый



Продолжение работы с матрицами. Многомерные массивы



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Введение

Основной блок

Задание для закрепления

Вопросы по основному блоку

Заключение



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Повторение

- Сортировка, изменение массивов
- Индексация для доступа к данным
- Двумерные массивы и изменение размерности
- Умножение матриц



Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Введение

Единичная матрица, операции над матрицами

Тензоры

Broadcasting

Добавление размерностей



Стать IT-шником может каждый



ОСНОВНОЙ БЛОК



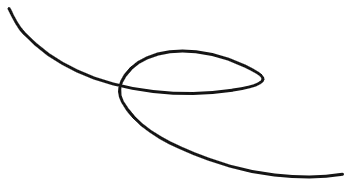
Единичная матрица, операции над матрицами





Единичная матрица —

это квадратная матрица, у которой на диагонали стоят единицы, а вне — нули. Свойство: при матричном умножении на единичную матрицу исходная матрица не изменится.



Экспресс-опрос



Объясните почему работает свойство: при умножении на единичную матрицу исходная матрица не изменится? Проверьте это свойство на примерах.



Реализация свойства

```
I = np.eye(4)
print(I)
# [[1. 0. 0. 0.]
#  [0. 1. 0. 0.]
#  [0. 0. 1. 0.]
#  [0. 0. 0. 1.]]
```

Проверка умножением

```
a = np.arange(16).reshape(4,4)
```

```
print(a)
```

```
print(a @ I)
```

```
# [[ 0  1  2  3]
```

```
# [ 4  5  6  7]
```

```
# [ 8  9 10 11]
```

```
# [12 13 14 15]]
```

```
# [[ 0.  1.  2.  3.]
```

```
# [ 4.  5.  6.  7.]
```

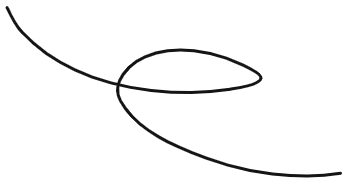
```
# [ 8.  9. 10. 11.]
```

```
# [12. 13. 14. 15.]]
```



fromfunction() —

это функция, которая применяет заданную функцию ко всем комбинациям индексов. Количество аргументов функции должно совпадать с размерностью желаемого массива.



Функция fromfunction()

```
def f(i, j):  
    print(i)  
    print(j)  
    return 10 * i + j  
  
print(np.fromfunction(f, (3, 4), dtype=np.int64))  
# [[0 0 0 0]  
# [1 1 1 1]  
# [2 2 2 2]]  
# [[0 1 2 3]  
# [0 1 2 3]  
# [0 1 2 3]]  
# [[ 0 1 2 3]  
# [10 11 12 13]  
# [20 21 22 23]]
```

Поиск файлов по дате изменений

hstack

Это функция, которая соединяет два массива в один. При этом у них должно быть одинаковое число строк, иначе выбрасывается ValueError

vstack

Это функция, которая соединяет два массива по вертикали, если у них одинаковое число столбцов

hsplit

Это функция, которая разделяет массив на несколько подмассивов по горизонтали (по столбцам)

Объединение массивов

```
a = np.array([1, 3, 2])
b = np.array([4, 5, 6])
a = np.hstack((a, b))
print(a)
# [ 1  3  2  4  5  6]

print(np.hsplit(a, [3, 6]))
# [array([1, 3, 2]), array([ 4,  5,  6]), array([], dtype=int32)]
```

Функции hstack и vstack

```
a = np.array([[0, 1], [2, 3]])  
b = np.array([[4, 5, 6], [7, 8, 9]])  
c = np.array([[4, 5], [6, 7], [8, 9]])  
print(a)  
print(b)  
print(c)  
# [[0 1]  
# [2 3]]  
# [[4 5 6]  
# [7 8 9]]  
# [[4 5]  
# [6 7]  
# [8 9]]
```

По горизонтали

```
print(np.hstack((a, b)))  
# [[0 1 4 5 6]  
#  [2 3 7 8 9]]
```

По вертикали

```
print(np.vstack((a, c)))  
# [[0 1]  
# [2 3]  
# [4 5]  
# [6 7]  
# [8 9]]
```

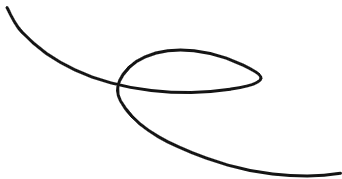
Сумма всех элементов

```
print(b)
print(b.sum())
print(b.sum(axis=0))
print(b.sum(axis=1))
# [[4 5 6]
#  [7 8 9]]
# 39
# [11 13 15]
# [15 24]
```



axis —

это аргумент, который показывает вдоль какой оси необходимо выполнять операцию. Это означает буквально направление, по которому нужно производить агрегацию. Аналогично работают `prod`, `max`, `min`, `mean` и т.д.



prod, max, min, mean

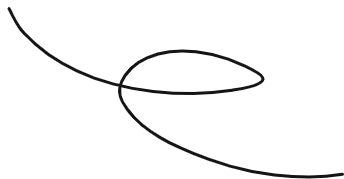
```
print(b.max())  
print(b.max(axis=0))  
print(b.min(axis=1))  
# 9  
# [7 8 9]  
# [4 7]
```

Полезно знать



След матрицы —

это **сумма** диагональных элементов.



След матрицы

```
print(np.trace(b))  
# 12
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Тензоры

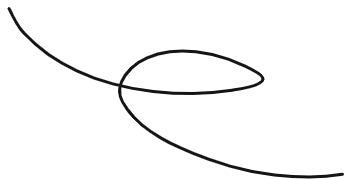


Полезно знать



Тензор —

это многомерный массив.



Тензор

```
X = np.arange(24).reshape(2, 3, 4)
print(X)
# [[[ 0  1  2  3]
#    [ 4  5  6  7]
#    [ 8  9 10 11]]
#
#    [[12 13 14 15]
#     [16 17 18 19]
#     [20 21 22 23]]]
```

Работа с осями

```
print(X.sum(axis=0))  
# [[12 14 16 18]  
#  [20 22 24 26]  
#  [28 30 32 34]]
```

Работа с осями

```
print(X.sum(axis=(1, 2)))  
# [ 66 210]
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Broadcasting



Перемножение массивов одинаковой формы

```
a = np.array([1, 2, 3])  
b = np.array([2, 2, 2])  
print(a * b)  
# [2 4 6]
```

Умножение элементов массива

```
print(a * 2)  
# [2 4 6]
```

Умножение элементов массива

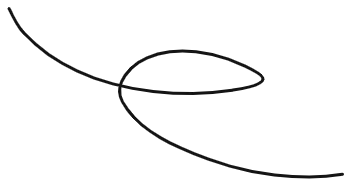
```
print(a * [2])  
# [2 4 6]
```

Полезно знать

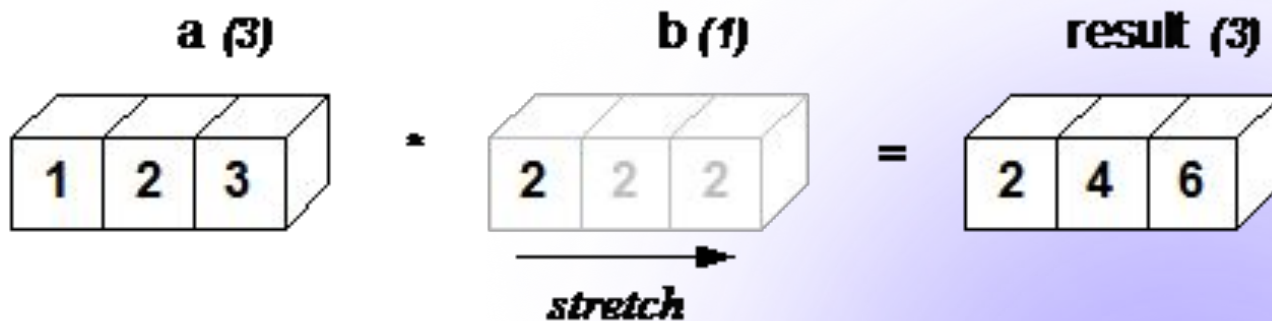


Broadcasting —

это процесс, когда один массив "растягивается", чтобы повторить форму другого.



Broadcasting

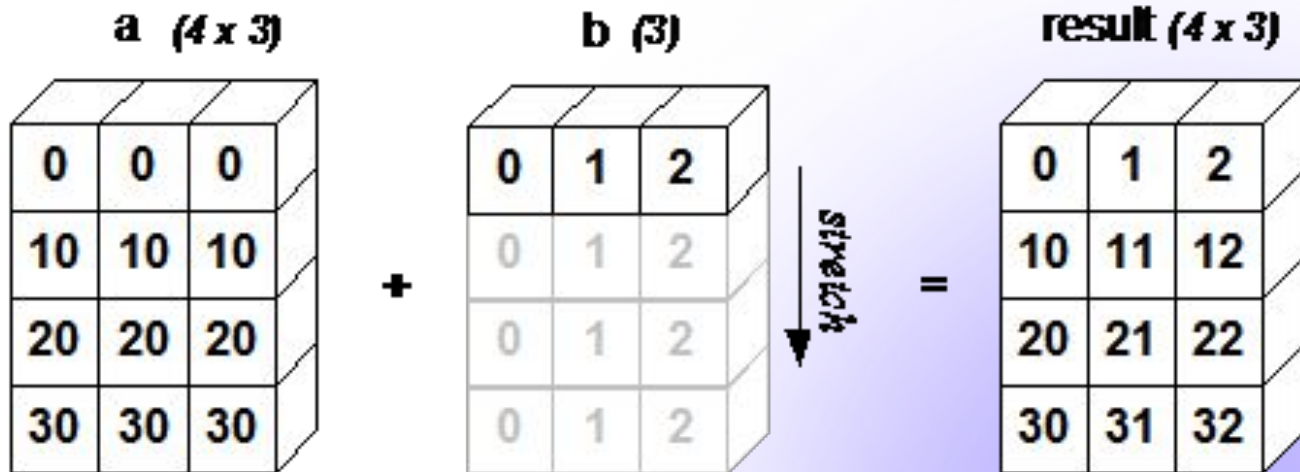


Broadcasting

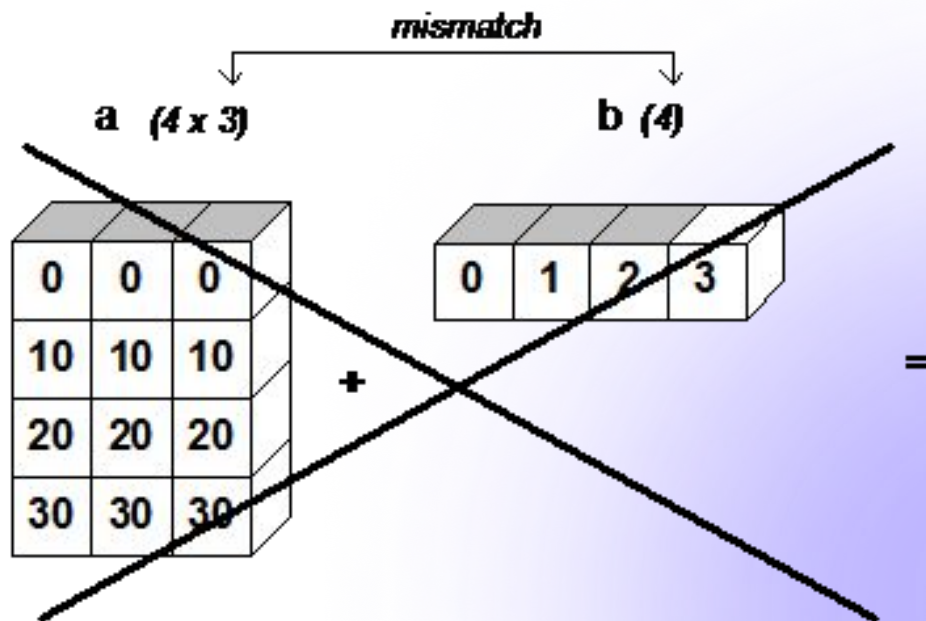
```
a = np.array([[ 0,  0,  0],  
             [10, 10, 10],  
             [20, 20, 20],  
             [30, 30, 30]])
```

```
b = np.array([0, 1, 2])  
print(a + b)  
# [[ 0  1  2]  
# [10 11 12]  
# [20 21 22]  
# [30 31 32]]
```

Broadcasting



Broadcasting



Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Подумайте, массив какого размера получится, если перемножить массив $4 \times 1 \times 3$ и массив 12×1 .
Убедитесь на практике в правильности вашего ответа.

```
# решение
a = np.ones((4, 1, 3))
b = np.ones((12, 1))

mul_shape = (a * b).shape
print(mul_shape)
# (4, 12, 3)
```

Стать IT-шником может каждый



Добавление размерностей

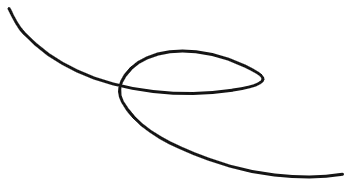


Полезно знать



newaxis —

это специальный объект, который позволяет добавлять новые оси.



Добавление размерностей

```
a = np.array([1,2,3,4,5,6,7,8,9,10])  
  
b = a[np.newaxis, :] # добавление оси axis0  
print(b.shape)  
# (1, 10)  
c = a[np.newaxis, :, np.newaxis]  
print(c.shape)  
# (1, 10, 1)
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Что будет выведено в следующем случае и почему:

```
import numpy as np

a = np.array([[0, 0, 0],
              [10, 10, 10],
              [20, 20, 20],
              [30, 30, 30]])
b = np.array([0, 1, 2])
result = a / (b + 1)
print(result)
```

Стать IT-шником может каждый



ПРАКТИЧЕСКАЯ РАБОТА



Практическое задание 1

Создайте два массива: первый - двумерный массив (матрица) размером 6x4, заполненный числами от 10 до 33; второй - одномерный массив длины 4 с числами от 15 до 18. Выполните поэлементное деление матрицы на одномерный массив, используя правила Broadcasting, и вычислите след результирующей матрицы.

Практическое задание 2

Создайте двумерный массив размером 3×8 , заполненный числами от 1 до 24. Выполните суммирование по первой оси ($\text{axis}=0$), посчитайте среднее значение по второй оси. Выведите полученный результат.

Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ





Домашнее задание

1. Напишите программу, которая:

- Запрашивает у пользователя ввод одномерного массива, элементы которого вводятся в виде чисел, разделенных пробелами.
- Преобразует введенную строку в массив NumPy.
- Запрашивает у пользователя индекс элемента, который необходимо извлечь из массива.
- Выводит значение элемента по запрошенному индексу.
- Запрашивает у пользователя ввод диапазона индексов в формате `начало:конец`, чтобы извлечь срез массива.
- Выводит срез массива.



Домашнее задание

2. Напишите программу, которая:

- Запрашивает у пользователя размеры матрицы $M \times N$.
- Генерирует две матрицы указанных размеров, заполненные случайными числами от 0 до 9.
- Выводит обе сгенерированные матрицы на экран.
- Выполняет и выводит результат одной из следующих операций (на выбор пользователя): сложение матриц, умножение матриц, транспонирование одной из матриц.
- Если выбрано умножение, убедитесь, что размеры матриц соответствуют правилам матричного умножения.

Полезные ссылки

- [Broadcasting — NumPy v1.26 Manual](#)
- [Array objects — NumPy v1.26 Manual](#)

Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

